Name	Student ID	Contribution
Amit Rathi	2025AF05169	100%
T R Sai Rakshith	2025AE05180	100%
Mohit Kashyap	2025AF05035	100%
Prashant Mhetar	2025AE05119	100%
Jyoti Kumari	2025AE05237	100%

Video Recording:

https://drive.google.com/file/d/1hhVg-y8jyDOoiuYbL0_AhWq6dbSQHB3o/view?usp=sharing

Github Link: https://github.com/prashantmhetar-bits/logi-edge

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Final Report
AIML ZG535 — Machine Learning on Edge · Continuous Evaluation Component EC-I
Birla Institute of Technology & Science, Pilani · Work Integrated Learning Programmes Division

1. FreightBridge Deployment Context

FreightBridge's pilot is 85 refrigerated trucks carrying medicines across Maharashtra, Karnataka and Andhra Pradesh. A refrigeration failure raises cargo temperature by about 1 °C per minute, so three minutes of silence already moves a 4 °C shipment from Warning into Critical. Detection and the alert must finish within 90 seconds, including on the Nashik–Aurangabad road, where coverage drops for 35 to 90 minutes at seven places. A cloud round trip of 50 to 500 ms is enough on a live link and useless in a gap, because no samples, inference or custody line exist until the modem returns. The edge node decides on the truck. Windowing uses most of the 90 seconds (about 50 seconds worst case, Section 4). INT8 inference is a fraction of a millisecond, so the alarm does not wait for a tower.

Each truck emits temperature at 1 Hz, three-axis vibration at 500 Hz, and door events. Temperature at 8 bytes a sample is 0.66 MB/day. Vibration at 12 bytes a sample is 43.2 million samples and 494 MB/day. One hundred door events add 0.003 MB. The raw total is 494.66 MB per truck per day: 49.47 per truck, 4,204 a day for 85 trucks, and about 15.34 lakh a year at 0.10/MB, before any model update. The node uploads an alert only on a state change. Two hundred alerts of 200 bytes are 0.04 MB/day, a 99.99 percent cut. In a gap the broker, the model and the alert log keep running. Alerts are fsync'd to disk before publish, and the bridge queues them until the link returns. The broker binds to 127.0.0.1. Only alerts and status cross the cellular link, under mutual TLS with a per-truck certificate, so clients receive a classified event rather than the sensor tape.

Three boards were scored on the Constraint Triangle of latency, power and unit cost. Linux, Docker, MQTT and Ansible sit on the cost vertex, because a board that cannot take an update is expensive at 85 trucks and again at 265. The Raspberry Pi 5 (8 GB) plus Hailo AI HAT+ draws 7.5 W, inside the 10 W budget from the 12 V truck feed, costs about 15,000, and runs that toolchain: 12.75 lakh for the pilot and 39.75 lakh for 265 trucks. The Jetson Orin Nano Super (67 TOPS, 15 W, 45,000) misses the power budget and costs 38.25 lakh for the pilot, 1.19 crore at full scale, for a model this small. The STM32H7 (3,500, 0.4 W) wins on power and price and loses deployment, because Docker and Ansible are not practical on it. The dominant vertex is enough compute for the 90-second SLA, balanced against unit cost, with 10 W as a filter. The Pi 5 passes. The Hailo chip stays for a later, heavier model; this network runs on the Cortex-A76.

The brief's Roofline inputs are 45 MFLOPs and 18 MB moved per inference, on a Pi 5 CPU of 16 GFLOP/s and 12 GB/s. Arithmetic intensity is 45 million / 18 million = 2.5 FLOPs/byte. The ridge point is 16 / 12 = 1.33 FLOPs/byte. The reference point sits above the ridge, so that workload is compute-bound, and the matching steps are INT8 arithmetic and structured pruning (M2 and M3). The trained network is smaller, about 1.5 thousand FLOPs in FP32 and 0.7 thousand after pruning, with a 0.53 KB working set, and it still ships as INT8 TensorFlow Lite on the CPU. Figure 1 shows sensors, local Mosquitto, the Pi 5 pipeline, the alert log, the uplink and the operations centre.

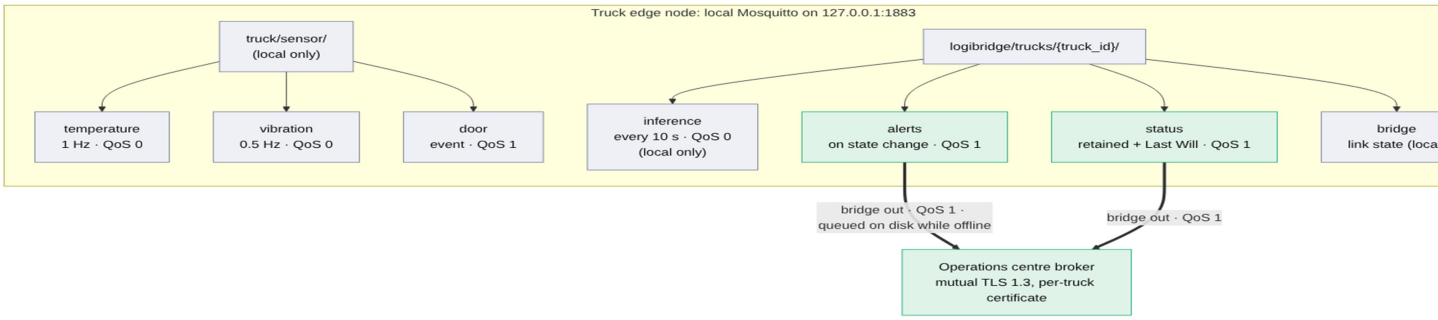


Figure 1. Sensing, inference and the alert log stay on the truck. The cellular link carries queued alerts after coverage returns. The drawing labels the local log as SQLite; the service writes alerts.jsonl on a Docker volume, which is the same store-and-forward role.

2. Sensor Pipeline and MQTT Design

Each truck runs Mosquitto on the loopback interface. The simulator, the inference service and the drift monitor use only that broker, so a dead modem does not stop classification. One outgoing bridge forwards the two topics the operations centre needs and holds them on disk while the link is down. Figure 2 is the tree the code publishes. Figure 1 uses a shorter prefix; the implementation uses the names below.

Figure 2. Topics on 127.0.0.1:1883. Green topics are bridged. Every other topic stays on the vehicle.

Topic	Rate	QoS	Leaves the truck?
truck/sensor/temperature	1 Hz	0	No
truck/sensor/vibration	0.5 Hz RMS	0	No
truck/sensor/door	per open or close	1	No
logibridge/trucks/{id}/inference	every 10 s	0	No
logibridge/trucks/{id}/alerts	on state change	1	Yes, queued
logibridge/trucks/{id}/status	connect and last will	1, retained	Yes

Temperature and vibration use QoS 0. They are frequent loopback streams, and a missed sample is absorbed by a five-sample average inside a 30-second window of about 30 temperature and 15 vibration points. QoS 1 would add a handshake to about 130,000 messages a day with no detection benefit. Door events use QoS 1: a lost OPEN or CLOSE corrupts the custody state, and a duplicate is harmless because the payload is timestamped. Inference uses QoS 0 and is not bridged. It feeds the drift monitor; one lost window moves a histogram bin by about one percent, under the 0.25 alert line, while bridging 8,640 messages a day would erase the saving in Section 1. Alerts use QoS 1, are bridged, and are queued with persistence, so an alert raised in a 35-to-90-minute gap is delivered when coverage returns, including after a reboot. QoS 2 would add a four-step handshake on a weak rural link. A sequence number lets the backend drop QoS 1 duplicates. Status is QoS 1, retained, with a last will, so a subscriber sees the current state immediately and a crashed process is shown offline. A silent truck must not look healthy.

Preprocessing follows a fixed order. Temperature and vibration pass a five-sample moving average. Every 10 seconds the service emits six features from the last 30 seconds: temperature mean, standard deviation and rate of change (°C/min), plus vibration RMS, peak and kurtosis. Door events stay on their own topic. They are not in the vector, because the specified features are these two sensors. Normalisation uses mean and standard deviation from ten minutes of clean Normal data, stored in training_stats.npy and loaded at runtime, never recomputed on the live window. Standard deviations are floored (0.1 on the tight Normal features, 0.2 on the rate of change, 1.0 on kurtosis) so a near-zero variance cannot explode the scale, and the vector is clipped to ±10.

The shift experiment scores the same 59-window holdout (24 Normal, 18 Warning, 17 Critical) after moving the stored mean by three standard deviations.

Statistics	M1 accuracy	M2 accuracy	M3 accuracy	What changed
Correct file	100%	100%	100%	Critical recall 100% on all three
Mean +3σ	98.3%	100%	98.3%	M1 misses 1 Critical; M3 misses 1 Warning
Mean -3σ	61.0%	61.0%	61.0%	23 of 24 Normal windows called Critical

A three-sigma error in the stored mean turns a healthy compartment into false Critical alarms. The statistics file is therefore shipped with the weights by the deployment playbook.

Fusion is at feature level: each sensor is reduced on its own, then the six numbers feed one classifier. Data-level fusion would resample 1 Hz against 0.5 Hz (and a 500 Hz accelerometer on the real truck) into a mixed-unit matrix much larger than M3's 0.53 KB working set. Decision-level fusion, two classifiers combined by a vote, would drop the joint pattern that separates Warning from Critical here. Both classes drift in temperature. Only the vibration step, about 0.45 g to 1.2 g RMS, marks the bearing-wear class. A temperature-only vote would call both Warning. A vibration-only vote would miss a pure drift. One model on the concatenated vector learns that interaction and still fits the device.

3. Model Deployment and Pipeline Mapping

LogiEdge follows the ten stages of the edge pipeline:

1. **Problem Framing**: Fixes a three-class compartment state, a 90-second alert, and a ban on any design that needs continuous cellular coverage.
2. **Data Acquisition**: A simulator on the local broker, switched between no anomaly, temperature drift, a vibration step, or both.
3. **Preprocessing**: The five-sample moving average applied before a window is cut.
4. **Features and Fusion**: Turn each 30-second window, stepped by 10 seconds, into the six-value vector of Section 2, scaled with the frozen statistics.
5. **Training**: Fits a perceptron with hidden widths 32 and 16. The committed files hold 117 Normal, 87 Warning, and 87 Critical windows, close to the requested 20, 15, and 15 minutes, and the saved run fitted 235 windows.
6. **Evaluation**: Holds out a stratified 20 percent, 59 windows (24 / 18 / 17), and refuses to save a model under 88 percent accuracy or 95 percent Critical recall. The FP32 model scores 100 percent on both.

7. **Optimisation**: Builds M2 by full-integer post-training quantisation and M3 by removing about 35 percent of the hidden neurons, then quantising.
8. **Conversion**: Writes both as TensorFlow Lite with integer weights, activations, and an integer input quantiser, calibrated on the training windows.
9. **Packaging**: Puts the service in a python:3.11-slim image. MODEL_PATH selects the file at start, and Ansible copies that file onto the node and restarts the container.
10. **Monitoring**: Runs a sidecar that computes the population stability index on the probability of Normal and alerts above 0.25. The playbook ships the reference histogram beside the weights.

The image is ordered so a model update is cheap. Every pip install, including TensorFlow, comes before the model-file copy, and that copy is the last filesystem layer. A model-only rebuild re-runs that layer and leaves the runtime cached on the truck. The three files measure 34.46 KB, 4.58 KB and 3.75 KB (43,819 bytes). At 0.10/MB one truck downloads 0.044 MB and 85 trucks download 3.72 MB, 0.37, instead of repeating the TensorFlow layer. MODEL_PATH still selects FP32, plain INT8 or pruned INT8 inside the same image.

Updates are every six weeks. The brief prices the file at 280 KB (0.2734 MB). A cycle that reaches all 85 trucks moves 23.24 MB and costs 2.32 for full replacement, canary and shadow alike. Trained M3 is 3.75 KB, about 75 times smaller, so 2.32 is a cap. Full replacement puts a bad file on every truck at once, including trucks inside a gap that cannot roll back until the next tower. Shadow mode keeps the old model in charge while the new one runs beside it, at the cost of a second inference, and six rural weeks may contain no Critical event, so silence looks like agreement. Canary is the recommendation: 10 trucks first (0.27), then the other 75 (2.05) if drift and Critical detections stay healthy. A finished cycle still costs 2.32, the blast radius is ten trucks, and a failed canary is reversed by re-running the playbook on the previous model path.

4. Optimisation and Pareto Analysis

M1 is the FP32 network (803 parameters). M2 is full INT8 post-training quantisation on the 235 training windows. M3 decays to 35 percent structured pruning, then uses the same INT8 step. Magnitude pruning zeroes weights and leaves the layers at width 32 and 16, so the integer file does not shrink. Neurons are ranked by incoming-weight energy, the smallest are dropped, and the net is rebuilt at widths 21 and 10 (400 parameters). It matches the masked outputs to within 10^-5. Timing is batch size one, ten warm-up runs discarded, 200 timed runs, on an i7-1255U (15 W TDP, AC power). Energy is that TDP times mean latency. Process CPU was about 5 to 8 percent and is not multiplied in.

Variant	Mean latency (ms)	p95 latency (ms)	Size (KB)	Accuracy (%)	Energy (mJ)

Variant	Mean latency (ms)	p95 latency (ms)	Size (KB)	Accuracy (%)	Energy (mJ)
M1 FP32	2.261	3.443	34.46	100	33.91
M2 PTQ INT8	0.0145	0.0149	4.58	100	0.217
M3 pruned + INT8	0.0200	0.0359	3.75	100	0.301

Table. Fifteen measurements on the 59-window holdout. Critical recall is 100 percent (17/17) on every row. M1's SavedModel is 51.43 KB. The M1 latency above is the forward pass; predict, as the service calls it, is 63.9 ms mean and 73.6 ms at p95.

Accuracy does not separate the models: INT8 matches FP32 on every validation window. Latency and energy put M1 more than a hundred times behind the integer models, and they leave M2 and M3 tied. Across repeats those two means sit between 0.010 and 0.022 ms and swap order. Size does not swap.

M3 is 18 percent smaller than M2 and uses 0.53 KB of weights and activations against 0.99 KB. Figure 3 plots accuracy against latency, size and energy. All three sit at 100 percent, above the 88 percent line. M1 falls inside the frontier on every panel. M2 and M3 both lie on it, M2 for energy in this run and M3 for file size. The stable tie-break is size, so M3 is deployed.

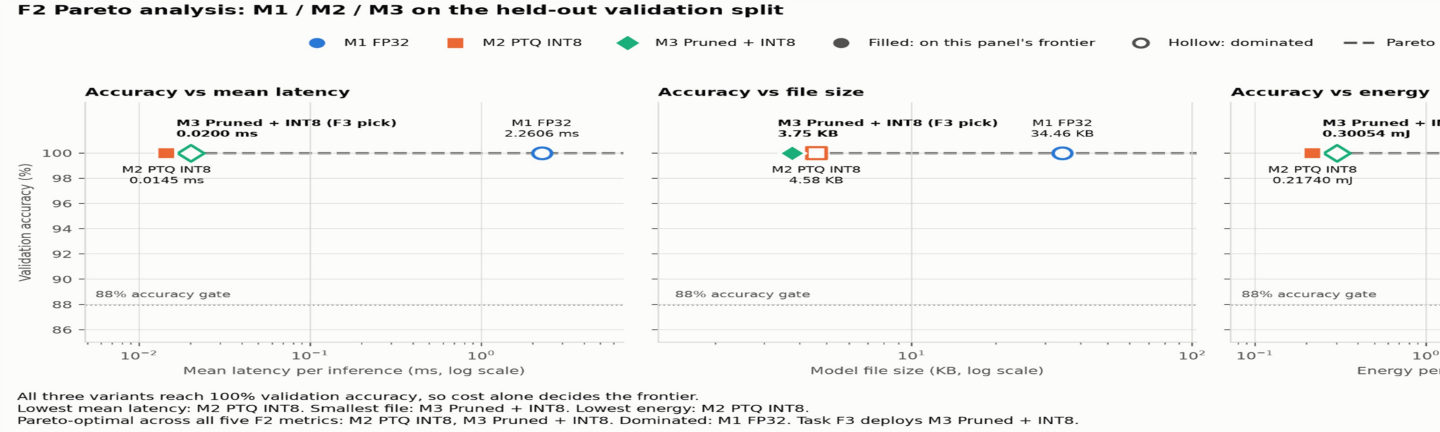


Figure 3. Filled markers are on that panel's frontier. M1 is dominated. M3 is the model recommended for the trucks.

Read as a per-inference budget, the SLA leaves little work for the model. Worst case, the vibration average takes 10 seconds to flush, the window takes 30 seconds to fill with post-fault data, and the fault can fall just after a step. That is 50 seconds, leaving 40. The service runs every 10 seconds on one thread, so an inference slower than 10 seconds would push the next alert past the

SLA. The engineering target is 100 ms on device. M3's laptop p95 is 0.036 ms, and a pessimistic tenfold slowdown on the Pi 5 is still about 0.4 ms. M1 through predict, scaled the same way, is the only variant that threatens 100 ms. M3's weights are 0.10 percent of the 512 KB L2 cache, and both integer models need only the TFLite interpreter.

Critical recall on M3 is 17/17, above the 95 percent gate, and conversion fails closed if it is not. One miss on this holdout would fall to 94.1 percent, and 17 clean detections support a 95 percent lower confidence bound near 84 percent. A bound above 95 percent needs at least 59 confirmed Critical events with none missed, which is a pilot exit criterion. Labelling is the other limit. Warning is temperature drift and Critical is that drift plus the vibration step, so temperature alone does not define the class. After pruning, M3 follows a rising temperature more than a high one, and the integer input range maps temperature mean only from about 3.74 °C to 5.01 °C, so a cold excursion is invisible to M2 and M3. The service applies a rule after the model: more than 1 °C from the 4 °C setpoint is at least Warning, and more than 3 °C away is Critical, on either side. The rule only raises the model's class. Combined Critical recall on the holdout stays 17/17.

Deploy M3 on the Pi 5 CPU with that rule, and keep M2 in the image as the rollback: a second playbook run or a MODEL_PATH change, with no new image. M1 is rejected on predict latency, file size and runtime weight.

5. MLOps Monitoring and Reflection

Drift is measured on the probability of Normal, the first entry of probabilities on the local inference topic. The reference is a four-bin histogram of that score on 300 clean windows, with edges at 0.25, 0.50 and 0.75. The sidecar keeps the last 100 scores, recomputes the population stability index every 60 seconds, and prints [LOGIBRIDGE DRIFT ALERT] above 0.25. Empty bins take a Jeffreys offset of one half so an unseen bin cannot send the index to infinity. Using that function, 300 reference scores in the top bin and a clean window of 100 scores in the same bin give 0.011. That is the value before injection while the model is confidently Normal. When the combined anomaly fills the window with scores in the bottom bin, the index rises to 11.55, and it crosses 0.25 as soon as a minority of the 100 scores leave the top bin, inside the five-minute limit at one inference every 10 seconds. Restoring clean data returns the index to 0.011 once the window has flushed. The recovery line used by the demo is 0.10.

The injected fault is a sudden covariate shift: a temperature ramp plus a vibration step. That is data drift, not a change in what the labels mean, and it appears as prediction drift because P(Normal) collapses when the model commits to Critical. Watching the maximum class probability would miss it. Under the combined fault the model is confidently Critical, the maximum stays in the top bin, and the index stays near 0.011. The score under test has to be the probability of the healthy class.

The playbook is seven tasks: create /opt/logibridge, copy model.tflite, copy reference_dist.json, stop the container only if a copy changed, pull the image, start it with MODEL_PATH and the truck

id, then wait 15 seconds and accept a running container whose restart count is zero. The default file is M3. A second run with nothing changed must report changed=0: copies are checksummed, the stop is skipped when both files match, and the health check is not a change. Continuous integration runs the play twice and fails the build otherwise.

The long step was pruning. Magnitude pruning left an integer file as wide as M2, because zeroed weights are still stored. Dropping whole neurons and checking the rebuilt net against the masked outputs is what made M3 the smallest file. Before a second pilot, windows should be labelled from the brief's temperature bands, including cold excursions, and the quantiser should cover those values. The temperature rule closes the gap here. It should not stay the only definition of Critical on real cargo.